

Exercícios Avaliativos

Programação Orientada a Objetos Intermediária e Avançada

Apresentação

Os exercícios apresentados a seguir fazem parte dos instrumentos de avaliação da disciplina de Programação Orientada a Objetos para o NAP I (Nota de Avaliação Parcial I). No caso, referem-se ao critério de avaliação denominado “exercícios em sala em dupla” e correspondem a 30% da nota final. No entanto, em razão do contexto atual de infraestrutura da universidade, o mesmo será aplicado em grupo, como tarefa do SIGAA.

Os exercícios devem ser respondidos em duas linguagens de programação, **Python** e **Java**. Cada solução corresponderá a um visto, certo ou errado. Então a nota final desse critério de avaliação corresponderá à proporção do número de acertos. Por exemplo:

- considerando que até o final do NAP I dez exercícios sejam aplicados
- então das 20 respostas esperadas, dez em Java e dez em Python, 18 respostas estejam corretas
- a nota final no critério “exercícios em sala em dupla” será 9 (nove)
 - $18/20 * 10 = 9$
- considerando que esse critério corresponde a 30% da nota final
 - $9 * 30\% = 2,7$
 - essa nota corresponderá a 2,7 pontos na nota final do NAP I

Para acompanhar o resultado dos vistos, visite a planilha compartilhada abaixo:

<https://docs.google.com/spreadsheets/d/13sydryYyFemS6yoQrE-mohAPCtvZixoF8pKJlhfmOs/edit?usp=sharing>

Orientações de Submissão da Resposta

A submissão da solução será através de uma tarefa do SIGAA, que pode ser acessada a partir do menu “Atividades >> Tarefas”. Nessa operação, a equipe deve submeter um único arquivo, com limite de tamanho de 10Mb, conforme restrições da plataforma. Assim, antes de submeter a tarefa pelo, cada equipe precisará seguir as orientações abaixo:

- Reunir todos os arquivos em **Java** produzidos para cada exercício, ou seja, classe da solução e mais a classe principal que manipula os objetos da solução
- Reunir todos os arquivos em **Python** produzidos para cada exercício, ou seja, arquivo da classe da solução e mais arquivo do programa principal que manipula os objetos da solução
- Compactar os arquivos é um único arquivo em formato ZIP, RAR ou 7z

A submissão da solução deve ser executada por apenas um membro da equipe. No caso, recomendamos que não esperem que todos os exercícios estejam respondidos para submeter a resposta, pois a tarefa no SIGAA estará configurada para permitir o envio de novas respostas.

Orientações Gerais

Atenção aos padrões de nomenclatura das classes e de seus atributos e métodos, que devem considerar o padrão [CamelCase](#), como segue

- Todos os atributos da classe são encapsulados
- Nome de classe iniciando com maiúsculas
- Nome de atributos e métodos iniciando com minúsculas
- Em Python, o nome do arquivo com o código da solução (classe ou programa principal) deve usar o padrão [SnakedCase](#)
- Em todas classes, com exceção da principal, adicionar método que retorna uma representação em String da mesma
 - Considerar, quando for o caso, definição específica do enunciado da questão
 - Não ausência dessa definição, a escolha dessa representação fica a critério do grupo

Ademais, considere que a solução de cada questão deve ser acompanhada de uma classe principal, que será responsável por ilustrar os objetos da solução, mesmo que essa orientação não esteja explicitamente indicada.

Exercício 1



Criar a classe **Pais**. Um país é representado através dos atributos:

- código ISO 3166-1 (ex.: BRA);
- nome (ex.: Brasil);
- população (ex.: 213.000.000);
- e a sua dimensão em Km² (ex.: 8.515.767,049).

Além disso, cada país mantém uma lista de outros países com os quais ele faz fronteira. A classe possui os métodos a seguir:

- Construtor que inicialize o código ISO, o nome e a dimensão do país;
- Métodos de acesso (getter/setter) para as propriedades código ISO, nome, população e dimensão do país;
- Um método que permita verificar se dois objetos representam o mesmo país (igualdade semântica);
 - Dois países são iguais se tiverem o mesmo código ISO.
- Um método que receba um país e o adicione à lista de países com fronteira
- Um método que receba um país e informe se é **limítrofe**;
- Um método que retorne a **densidade populacional** do país;
- Um método que receba um país como parâmetro e retorne a lista de **vizinhos comuns** aos dois países.

Considere uma classe principal que ilustra a criação de todos os países da América Latina que são fronteira com a região sul e centro oeste do Brasil. Então monte uma lista desses países e percorra essa lista imprimindo nome e densidade populacional.

Exercício 2

Criar a classe **Pessoa** que representa uma pessoa numa árvore genealógica. A pessoa possui um nome, um **pai** e uma **mãe** (que também são pessoas). Forneça os seguintes membros para a classe:

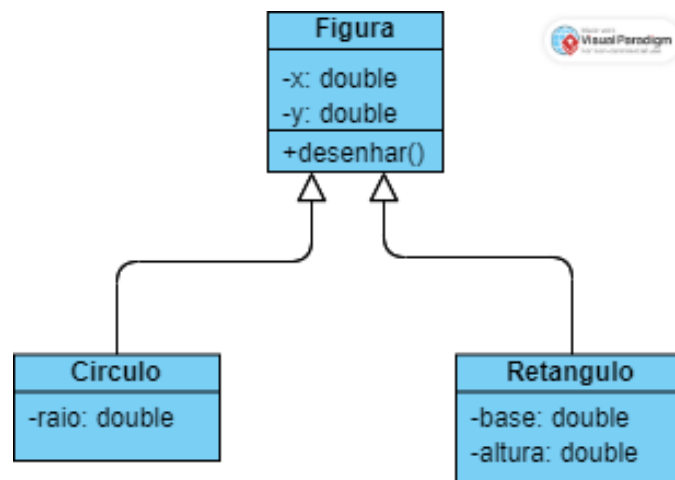
- a) Construtores sobrecarregados que:
 - i) inicialize o nome da pessoa, bem como seus ancestrais (pai e mãe);
 - ii) inicialize o nome da pessoa, sem informar seus ancestrais (null);
- b) Um método que verifique a **igualdade semântica** entre duas pessoas:
 - i) as pessoas são iguais se possuem o mesmo nome e os mesmos pais.
- c) Um método que verifique se duas pessoas são irmãs:
 - i) ou possuem a mesma mãe;
 - ii) ou possuem o mesmo pai.
- d) Um método que verifique se uma pessoa é seu ancestral:
 - i) é seu pai ou sua mãe;
 - ii) ou ancestral do pai ou ancestral da mãe.



Considere uma classe principal que instancia a árvore genealógica de Jesus Cristo acima. Então, demonstrar no código:

- Instanciação usando os diferentes tipos de construtor (sobrecarregados)
- Se Joaquim é ancestral de Jesus Cristo
- Se Ana é ancestral de Jesus Cristo
- Se Jacó é ancestral de Jesus Cristo
- Se Maria é ancestral de Jesus Cristo
- Se José é ancestral de Jesus Cristo
- Se Joaquim é irmão de Jacó
- Se Jesus é irmão de Maria

Exercício 3

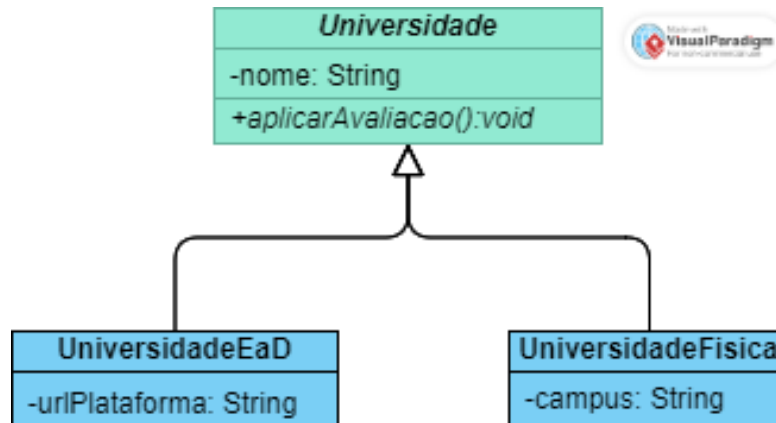


Criar as classes do diagrama acima. Os métodos de acesso (getter / setter) foram omitidos.

- a) Implementar em cada classe apenas um construtor **não padrão** que inicializa todos os atributos do respectivo objeto
 - i) No construtor da superclasse imprimir “Criando figura”
 - ii) No construtor da subclasse **Circulo** imprimir “Criando círculo”
 - iii) No construtor da subclasse **Retangulo** imprimir “Criando retângulo”
- b) O método **desenhar()** na classe **Figura** imprime a mensagem
 - i) “Desenhando uma figura na posição (x, y)”
 - ii) x e y são os valores das coordenadas da figura
- c) Na classe **Circulo**, sobrescrever o método **desenhar()** que imprime
 - i) “Desenhando um círculo na posição (x, y) com raio de r”
 - ii) x e y são os valores das coordenadas da figura e r o raio do círculo
- d) Na classe **Retangulo**, sobrescrever o método **desenhar()** que imprime
 - i) “Desenhando um retângulo na posição (x, y) com base b e altura a”
 - ii) x e y são os valores das coordenadas da figura e b e a são a base e a altura do retângulo

Considere uma classe principal que instancie e inicialize três objetos figura, um de cada tipo: **Figura**, **Circulo** e **Retangulo**. Então, demonstre a atribuição de objetos do tipo **Circulo** e **Retangulo** a uma variável do tipo **Figura**, bem como a chamada do método **desenhar** em cada objeto criado.

Exercício 4



Criar as classes do diagrama acima. Universidade é uma classe abstrata, que possui um método abstrato **aplicarAvaliacao()**, enquanto UniversidadeEaD e UniversidadeFisica são especializações concretas da classe abstrata. Os métodos de acesso (getter e setter) foram omitidos no diagrama, mas devem ser implementados.

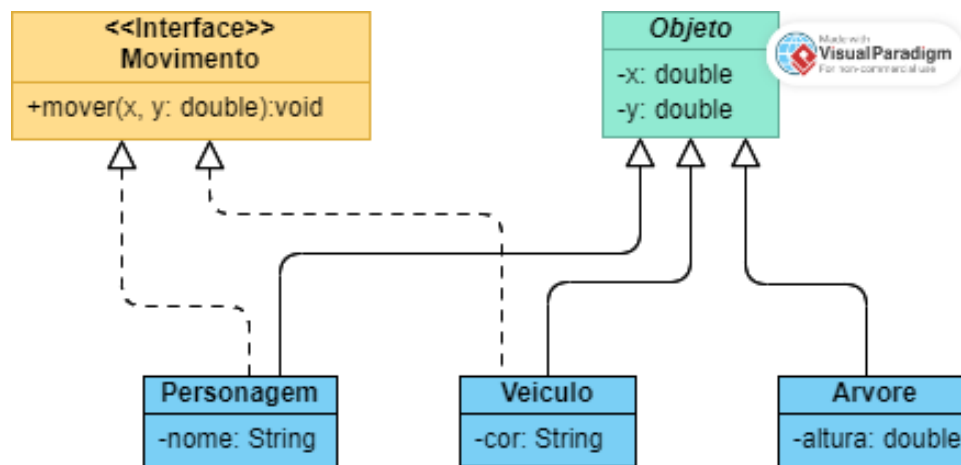
O método **aplicarAvaliacao()** deve apenas imprimir a seguinte mensagem:

- a) Em UniversidadeEaD imprimir
 - i) “Avaliações Online na url”
 - ii) url é o valor do atributo urlPlataforma
- b) Em UniversidadeFisica imprimir
 - i) “Avaliações presenciais no campus campus”
 - ii) campus é o valor do atributo campus

Considere uma classe/programa principal que:

- Crie uma lista de universidades
- Popule essa lista com pelo menos quatro objetos
 - Considere instanciar objetos UniversidadeEaD e UniversidadeFisica
- Para cada objeto da lista
 - Imprimir o nome da universidade;
 - E chamar o método aplicarAvaliacao().

Exercício 5



Criar as classes do diagrama acima, que implementam um jogo com personagens, veículos e árvores. Então, considere as orientações abaixo:

- Crie as classes e a interface do diagrama
- Objeto é uma classe abstrata que representa os objetos do jogo e identifica a localização dos mesmos (coordenadas x e y)
- Personagem, Veiculo e Arvore são classes concretas de Objeto
- Movimento é uma interface que define o método `mover(x, y)`, indicando os objetos que podem se movimentar no jogo
 - Atribui os argumentos x e y no objeto
 - Personagem e Veículo podem se mover
- Árvore não pode se mover no jogo pois não implementa Movimento
- Sobrescreva o método de representação do objeto como uma String em todas as classes do diagrama (Objeto, Personagem, Veiculo e Arvore)
 - Usar todos os atributos do objeto na representação como String
 - Em subclasses considerar os atributos da superclasse Objeto
 - Em Java, sobrescrever `public String toString()`
 - Em Python, sobrescrever `def __str__(self)`

Criar uma classe principal com as seguintes orientações:

- Crie uma lista de objetos do jogo
 - Popule a lista com pelo menos seis instâncias de Objeto
 - Considere instanciar Personagem, Veiculo e Arvore
 - Imprima cada objeto da lista
- Crie uma lista de Movimento
 - Popule a lista com pelo menos três instâncias de Movimento
 - Considere instanciar Personagem e Veiculo
 - Para cada objeto da lista chamar o método `mover(x, y)`
 - Considere usar valores de x e y **aleatoriamente** gerados
 - Imprima o objeto movido